

Разработка скрипта для автоматизации установки и настройки программного обеспечения

Введение

Итак, необходимо провести исследование и реализация методов автоматизации процессов установки и настройки программного обеспечения. Автоматизация процессов установки и настройки программного обеспечения позволяет экономить много времени, ведь установка десятков программ вручную может занять часы работы. Скрипт же выполнит эту задачу полностью самостоятельно, без вмешательства пользователя. При этом ничего не мешает использовать скрипт сразу на множестве машин, например, в аудитории. Тогда вся работа займёт суммарно время, необходимое для запуска скрипта на каждом компьютере по отдельности, и время на выполнение скрипта (на компьютере, на котором скрипт в последнюю очередь) – условно, час в спокойном темпе вместо 10 часов относительно интенсивной работы.

Исследование

По сути, выбор стоял между использованием .bat-файла и .ps1-файла. Также рассматривались способы упростить выполнение поставленной задачи автоматизации установки программного обеспечения путём использования пакетных менеджеров (Chocolatey, WinGet). В ходе работы были испробованы разные варианты выполнения задания, и остановился я на .ps1 + Chocolatey. PowerShell мощнее и современнее batch-файлов, а Chocolatey имеет большой репозиторий пакетов. В итоге был создан скрипт для автоматизации установки и настройки определённого программного обеспечения на Windows.

Важные нюансы

- Самым главным условием работы скрипта является наличие прав администратора! (Нужно запустить PowerShell от имени администратора и в нём уже запустить скрипт)
- Скрипт выполняется отнюдь не быстро, а скорость его выполнения зависит в первую очередь от скорости интернета
- Некоторое программное обеспечение (например, Docker) может потребовать перезагрузить систему после установки

Работоспособность скрипта была проверена на практике, как результат: успешная установка всего программного обеспечения, входящего в скрипт. В качестве примера был сделан скриншот в произвольный момент времени работы скрипта (Рис. 1).

```
Installing the following packages:
7zip
By installing, you accept licenses for the packages.
Downloading package from source 'https://community.chocolatey.org/api/v2/'

7zip.install v25.1.0 [Approved]
7zip.install package files install completed. Performing other installation steps.
Installing 64 bit version
Installing 7zip.install...
7zip.install has been installed.
7zip installed to 'C:\Program Files\7-Zip'
Added C:\ProgramData\chocolatey\bin\7z.exe shim pointed to 'c:\program files\7-zip\7z.exe'.
7zip.install can be automatically uninstalled.
The install of 7zip.install was successful.
Deployed to 'C:\Program Files\7-Zip\'
  Downloading package from source 'https://community.chocolatey.org/api/v2/'

    7zip v25.1.0 [Approved]
    7zip package files install completed. Performing other installation steps.
    The install of 7zip was successful.
    Deployed to 'C:\ProgramData\chocolatey\lib\7zip'

Chocolatey installed 2/2 packages.
See the log for details (C:\ProgramData\chocolatey\logs\chocolatey.log).
Downloading vlc...
Chocolatey v2.5.1
3 validations performed. 2 success(es), 1 warning(s), and 0 error(s).

Validation Warnings:
- A pending system reboot request has been detected, however, this is
  being ignored due to the current Chocolatey configuration. If you
  want to halt when this occurs, then either set the global feature
  using:
    choco feature enable --name="exitOnRebootDetected"
  or pass the option --exit-when-reboot-detected.

Installing the following packages:
vlc
By installing, you accept licenses for the packages.
Downloading package from source 'https://community.chocolatey.org/api/v2/'

vlc.install v3.0.21 [Approved]
vlc.install package files install completed. Performing other installation steps.
Installing 64-bit vlc.install...
vlc.install has been installed.
```

Рисунок 1 – Работа скрипта в действии

Листинг кода

```
# Automatic software installation script for Windows

# Disables the confirmation request for Chocolatey
Set-ExecutionPolicy Bypass -Scope Process -Force

# Chocolatey installation
Write-Host "Downloading Chocolatey..." -ForegroundColor Green
[System.Net.ServicePointManager]::SecurityProtocol =
[System.Net.ServicePointManager]::SecurityProtocol -bor 3072
iex ((New-Object
System.Net.WebClient).DownloadString('https://community.chocolatey.org/install.ps
1'))
refreshenv

# Installing software using Chocolatey

# The list of software that can be installed via Chocolatey
$chocoPackages = @(
    "vscode",
    "docker-desktop",
    "pycharm-community",
    "git",
    "github-desktop",
    "maxima",
    "gimp",
    "python",
    "msys2",
    "miktex",
    "texstudio",
    "anaconda3",
    "far",
    "sumatrapdf",
    "googlechrome",
    "firefox",
    "7zip"
)

Write-Host "Downloading Software through Chocolatey..." -ForegroundColor Green
foreach ($package in $chocoPackages) {
    Write-Host "Downloading $package..." -ForegroundColor Yellow
    choco install $package -y --no-progress
}

# Installing extensions for VS Code

Write-Host "Downloading extensions for VS Code..." -ForegroundColor Green

# The list of extensions for VS Code
$vscodeExtensions = @(
    "ms-python.python",
```

```

    "ms-vscode.cpptools",
    "ms-azuretools.vscode-docker",
    "ms-vscode.PowerShell",
    "yzhang.markdown-all-in-one",
    "bungcip.better-toml",
    "bung87.rust-analyzer",
    "julialang.language-julia"
)
foreach ($extension in $vscodeExtensions) {
    Write-Host "Downloading $extension extension..." -ForegroundColor Yellow
    code --install-extension $extension --force
}

# Installing software that is not available in Chocolatey (via direct links)
Write-Host "Installing software via direct links..." -ForegroundColor Green

# Julia installation
Write-Host "Downloading Julia..." -ForegroundColor Yellow
$ juliaUrl = "https://julialang-s3.julialang.org/bin/winnt/x64/1.10/julia-1.10.3-win64.exe"
$ juliaInstaller = "$env:TEMP\julia-installer.exe"
Invoke-WebRequest -Uri $juliaUrl -OutFile $juliaInstaller
Start-Process -FilePath $juliaInstaller -Args "/S" -Wait -Verb RunAs

# Rust installation
Write-Host "Downloading Rust..." -ForegroundColor Yellow
Invoke-WebRequest -Uri "https://win.rustup.rs/x86_64" -OutFile "$env:TEMP\rustup-init.exe"
Start-Process -FilePath "$env:TEMP\rustup-init.exe" -Args "-y" -Wait

# Zettlr installation
Write-Host "Downloading Zettlr..." -ForegroundColor Yellow
$ zettlrUrl = "https://github.com/Zettlr/Zettlr/releases/download/v3.0.4/Zettlr-Windows-x64-3.0.4.exe"
$ zettlrInstaller = "$env:TEMP\zettlr-installer.exe"
Invoke-WebRequest -Uri $zettlrUrl -OutFile $zettlrInstaller
Start-Process -FilePath $zettlrInstaller -Args "/S" -Wait -Verb RunAs

# Flameshot installation
Write-Host "Downloading Flameshot..." -ForegroundColor Yellow
$ flameshotUrl = "https://github.com/flameshot-org/flameshot/releases/download/v13.1.1/flameshot-13.1.1-win64-installer.exe"
$ flameshotInstaller = "$env:TEMP\flameshot-installer.exe"
Invoke-WebRequest -Uri $flameshotUrl -OutFile $flameshotInstaller
Start-Process -FilePath $flameshotInstaller -Args "/S" -Wait -Verb RunAs

# WSL2 and Ubuntu 22.04 installation
Write-Host "Downloading WSL2 and Ubuntu 22.04..." -ForegroundColor Yellow
wsl --install -d Ubuntu-22.04 --no-distribution-code

Write-Host "Installation completed! Please restart your computer to activate WSL2." -ForegroundColor Green

```